

Introduction to Kubernetes

Class no:1

Kubernetes

Kubernetes (also known as **K8s**) is an open-source container orchestration platform. Think of it as the conductor of an orchestra, where your software containers (like Docker) are the musicians. Instead of managing each container manually, Kubernetes automates their deployment, scaling, management, and networking so everything runs smoothly without human intervention.

Here is a breakdown of how it works and the key components that make it tick.

The Architecture: Control Plane vs. Worker Nodes

A Kubernetes setup is called a **Cluster**. A cluster is divided into two main parts: the **Control Plane** (the brains) and the **Worker Nodes** (the muscle).

1. The Control Plane (The Brains)

The Control Plane makes global decisions about the cluster (like scheduling applications) and detects/responds to cluster events.

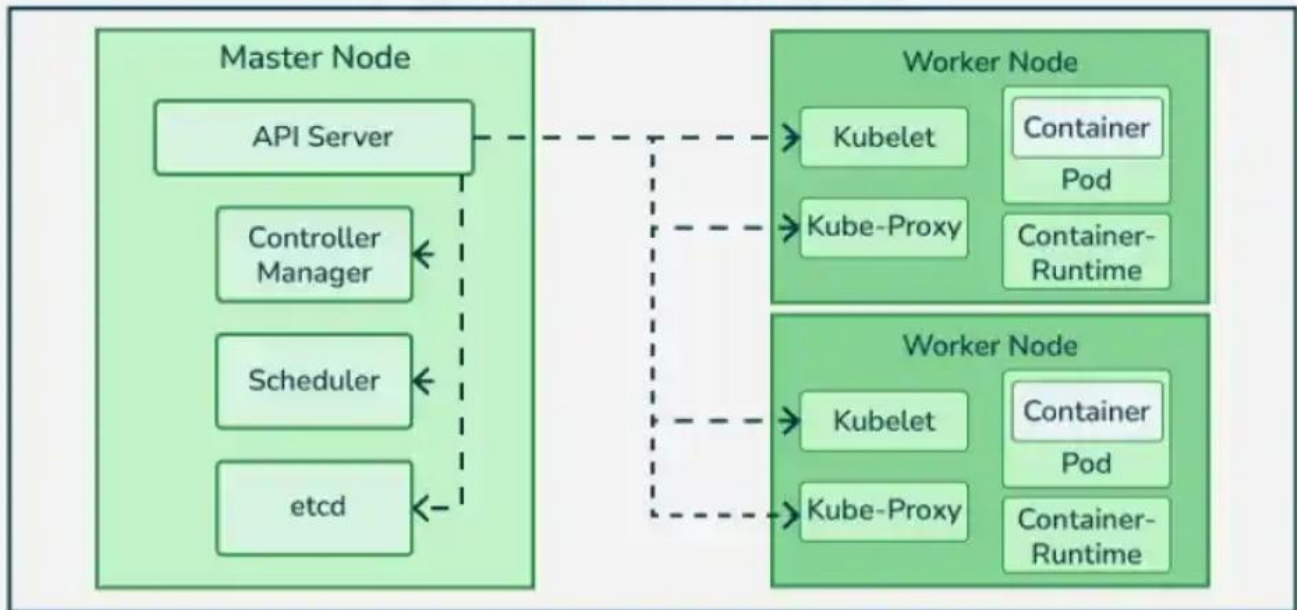
- **kube-apiserver:** The front door to Kubernetes. It exposes the Kubernetes API and is the hub that all other components talk to. Whether you use a command-line tool (like kubectl) or a dashboard, you are talking to this component.
- **etcd:** The cluster's memory. It is a highly available, distributed key-value store that holds all the configuration data and the exact state of everything in the cluster.
- **kube-scheduler:** The matchmaker. It watches for newly created containers (Pods) that have no node assigned to them, and selects the best Worker Node for them to run on based on resources, policies, and constraints.
- **kube-controller-manager:** The regulator. It runs controller processes in the background to regulate the state of the cluster. For example, if a container crashes, the Node Controller notices and ensures a new one is spun up to match your desired setup.
- **cloud-controller-manager:** The bridge to the cloud. This links your cluster into your cloud provider's API (like AWS, Google Cloud, or Azure) to manage things like cloud-based load balancers and storage volumes.

2. Worker Nodes (The Muscle)

Worker Nodes are the actual machines (virtual or physical) where your applications live and run.

- **Kubelet:** The captain of the ship on each node. It is an agent that runs on every worker node, ensuring that the containers are running exactly how the Control Plane requested. If a container goes down, the Kubelet tries to restart it.
- **Kube-proxy:** The traffic cop. It handles the network communication inside and outside of your cluster, making sure network traffic finds its way to the correct container.
- **Container Runtime:** The engine. This is the software responsible for actually running the containers. Kubernetes supports several runtimes, with **containerd** and **CRI-O** being the most common.

Kubernetes Architecture



Core Objects You Actively Manage

While the components above run the infrastructure, these are the actual building blocks you deploy into Kubernetes:

- **Pods:** The smallest deployable units in Kubernetes. A Pod represents a single instance of a running process in your cluster and contains one or more tightly coupled containers.
- **Services:** Since Pods can die and be replaced frequently, their IP addresses change. A Service acts as a permanent pointer (a stable IP or DNS name) that routes traffic to your Pods, serving as an internal load balancer.
- **Deployments:** This is where you declare the *desired state* of your application (e.g., "I want 3 copies of my web app running at all times"). The deployment controller takes care of updating or scaling your containers automatically.
- **ReplicaSet** is a core Kubernetes object whose sole job is to maintain a stable set of identical Pods running at any given time.

Think of it as a **guaranteed backup and scaling manager**. If you tell Kubernetes, *"I always want exactly 3 identical copies of my API running,"* the ReplicaSet is the component that rolls up its sleeves to make sure that happens.

How a ReplicaSet Works

A ReplicaSet monitors your cluster constantly and uses three key elements to do its job:

1. **Selectors:** It uses labels to identify which Pods it is responsible for managing.
2. **Replicas:** The defined number of Pods that *should* be running (e.g., replicas: 3).
3. **Pod Template:** The blueprint it uses to stamp out new Pods if it needs to create more.

If a worker node crashes and takes two of your Pods down with it, the ReplicaSet instantly notices that the actual count (1) doesn't match the desired count (3). It immediately uses the Pod template to spin up 2 brand-new Pods on a healthy node to bring the count back to 3.

Conversely, if you manually create an extra identical Pod, the ReplicaSet will see that you now have 4 Pods instead of 3, and it will terminate one to maintain the balance.

ReplicaSet vs. Deployment (The Gold Standard)

While ReplicaSets are incredibly useful, you will rarely create them directly. Instead, you will almost always use a **Deployment**.

Here is the hierarchy:

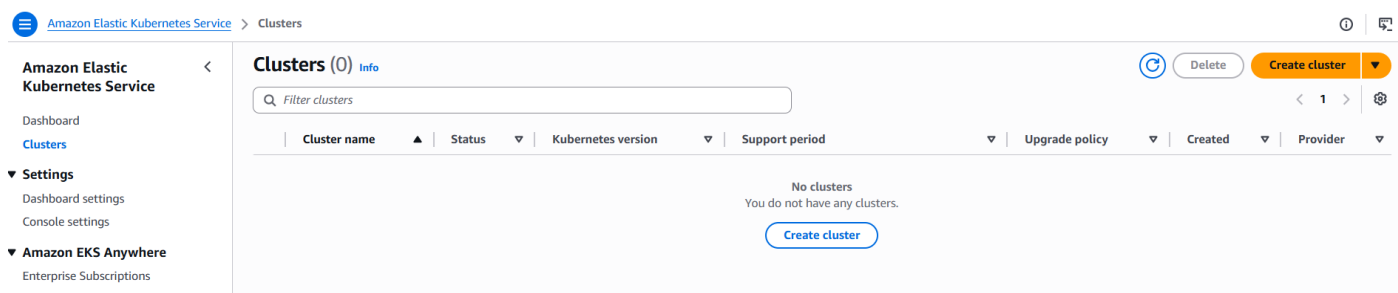
- **Deployment:** Manages the rollout of your application. When you update your application's code or image version, the Deployment automatically creates a *new* ReplicaSet with the new code, scales it up, and scales the *old* ReplicaSet down. This ensures zero-downtime updates.
- **ReplicaSet:** Manages the lifecycle and exact number of the individual Pods.
- **Pod:** The actual container running your code.

Analogy: If a **Pod** is a single worker, a **ReplicaSet** is the shift supervisor making sure the exact right number of workers are on the floor. A **Deployment** is the regional manager, deciding when it's time to train the workers on a brand-new menu or system.

Hands on: Creating a Kubernetes architecture via Ec2 and implementing its components:

Step no:1 To create a K8s architecture we need a need a cluster. Here in AWS we use **EKS cluster**.

Search **EKS** → **create cluster**.



Configuration: **custom configuration**

Name: **mycluster**

It will request for an **IAM role** in order to communication with the other AWS resources.

Create role and it will provide the policy needed for the role. Create and choose it.

Configuration options [Info](#)

Choose how you would like to configure the cluster.

☐ Quick configuration (with EKS Auto Mode)
Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

☒ Custom configuration
To change default settings prior to creation, choose this option. This configuration uses EKS Auto Mode and customizes the cluster's configuration.

EKS Auto Mode [Info](#)

Choose if you would like to use EKS's Auto Mode.

☒ Use EKS Auto Mode
EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

► Included capabilities

Cluster configuration [Info](#)

Name

Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

mycluster

The cluster name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Cluster IAM role

To access other AWS services and resources your Kubernetes control plane needs an IAM role. Choose a role with the right permissions for your Kubernetes control plane.

✓ AmazonEKSClusterRole successfully created.

AmazonEKSClusterRole



Create new role

Create role and it will provide the policy needed for the role. Create and choose it. → Next

Create role

Role details

Role name

AmazonEKSClusterRole

Maximum 64 characters. Use alphanumeric and +, -, @, _ characters.

Use case

This role will allow your EKS Kubernetes control plane to call AWS services on your behalf. [View trust policy.](#)

Default policies (1)

These policies will be attached to the role and define what permissions your EKS Kubernetes control plane has to access or create resources.

Policy name ↗



Type



[AmazonEKSClusterPolicy](#)

AWS managed

Keep the **version settings default** and move next.

Kubernetes version settings

Kubernetes version | [Info](#)
Select Kubernetes version for this cluster.

1.35

Upgrade policy | [Info](#)
Choose one of the following options. You can switch the setting later while the standard support period is in effect.

☒ **Standard support**
This option supports the Kubernetes version for 14 months after the release date. There is no additional cost. When standard support ends, your cluster will be auto upgraded to the next version.

☐ **Extended support**
This option supports the Kubernetes version for 30 months after the release date. This option has an additional hourly cost that is applied when support ends, your cluster will be auto upgraded to the next version.

Control plane scaling tier | [Info](#)
Select a scaling tier for your control plane. While Standard mode scales dynamically, higher tiers pre-provision your environment with fixed, high-performance capacity for demanding workloads. [View tier pricing](#)

☐ **Use a scaling tier**
For predictable and high performance from your cluster's control plane, choose a scaling tier.

In cluster access choose: **EKS API and ConfigMap**

Cluster access | [Info](#)
Control how IAM principals can access this cluster.

Bootstrap cluster administrator access | [Info](#)
Choose whether the IAM principal creating the cluster has Kubernetes cluster administrator access.

☒ **Allow cluster administrator access**
Allow cluster administrator access for your IAM principal.

☐ **Disallow cluster administrator access**
Disallow cluster administrator access for your IAM principal.

Cluster authentication mode | [Info](#)
Configure which source the cluster will use for authenticated IAM principals.

☐ **EKS API**
The cluster will source authenticated IAM principals only from EKS access entry APIs.

☒ **EKS API and ConfigMap**
The cluster will source authenticated IAM principals from both EKS access entry APIs and the aws-auth ConfigMap.

In Network settings,

Choose **default VPC** , all the subnets under in and **default SG** (ensure all TCP is enabled).

The **endpoint URL must be public**, so that we can see it from outside the console access as well.

Specify networking

Networking | [Info](#)
IP address family and service IP address range cannot be changed after cluster creation.

VPC | [Info](#)
Select a VPC to use for your EKS cluster resources.

vpc-0538c231a226fe3c9 | Default

Subnets | [Info](#)
Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster.

Select subnets

subnet-01bf4d440c0912db3
ap-south-1a 172.31.32.0/20

subnet-0c305b80f72541592
ap-south-1c 172.31.16.0/20

subnet-0b3c21b6de4314d95
ap-south-1b 172.31.0.0/20

Additional security groups | [Info](#)
EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optimize network performance by creating additional security groups for your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups

sg-06d80e515c32b9515 | default
default VPC security group

Choose cluster IP address family | [Info](#)
Specify the IP address type for pods and services in your cluster.

☒ **IPv4**

☐ **IPv6**

Cluster endpoint access [Info](#)

Configure access to the Kubernetes API server endpoint.

- ☒ **Public**
The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.
- ☐ **Public and private**
The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.
- ☐ **Private**
The cluster endpoint is only accessible through your VPC. Worker node traffic to the endpoint will stay within your VPC.

► **Advanced settings**

Cancel

Previous

Next

Monitoring: **default** , Add-ons (plugins of K8s): **default** → Next

Select add-ons

Review the add-ons from multiple categories, then select add-ons to enhance your cluster.

AWS add-ons (23) [Info](#)



Node monitoring agent [Info](#)

Enable automatic detection of node health issues.

Category
observability

Compatible compute
EC2, Hybrid Nodes



CoreDNS [Info](#)

Enable service discovery within your cluster.

Category
networking

Compatible compute
EC2, Hybrid Nodes, Fargate, EKS Auto Mode



Amazon VPC CNI [Info](#)

Enable pod networking within your cluster.

Category
networking

Compatible compute
EC2



kube-proxy [Info](#)

Enable service networking within your cluster.

Category
networking



Amazon SageMaker HyperPod observability

The HyperPod observability add-on installs multiple open-source metrics exporters and collectors to provide in-depth visibility into all generative AI tasks



Amazon SageMaker Spaces

With Spaces, you can create and manage JupyterLab and Code Editor applications to run interactive ML workloads.

Category

Review and click Create.

Review and create

Step 1: Cluster

Cluster configuration

Name
mycluster

Kubernetes version
1.35

EKS Auto Mode
Disabled

Upgrade policy
Standard support

Cluster IAM role
arn:aws:iam::314383175192:role/AmazonEKSClusterRole

Kubernetes cluster administrator
Allow cluster administrator access

Authentication mode
EKS API and ConfigMap

Control plane scaling tier
Standard

ARC Zonal shift

ARC Zonal shift

It will take 7 to 8 minutes to create the cluster.

Step no:2 In order to access the cluster we need an instance. This is for communication.

This instance is not the controller. The AWS will take care of the controlling by itself.

Create an **Amazon Linux instance** with a **key default VPC** and **default SG** (All TCP enabled).

Launch instance.

Name and tagsInfo

Name

EKS

Add additional tags

▼ Application and OS Images (Amazon Machine Image)Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use tBrowse more AMIs.

Q Search our full catalog including 1000s of application and OS images

RecentsQuick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Canonical Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI

ami-0db56f446d44f2f09 (64-bit (x86), uefi-preferred) / ami-07c18f602ca3306b9 (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimi to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.12.20260608.0 x86_64 HVM kernel-6.1

Architecture

64-bit (x86)

Boot mode

uefi-preferred

AMI ID

ami-0db56f446d44f2f09

Publish Date

2026-06-04

Username

ec2-user

▼ Instance typeInfo | Get advice

Instance type

t3.small

Family: t3 2 vCPU 2 GiB Memory Current generation: true

On-Demand Ubuntu Pro base pricing: 0.0259 USD per Hour On-Demand Linux base pricing: 0.0224 USD per Hour

On-Demand SUSE base pricing: 0.0534 USD per Hour On-Demand RHEL base pricing: 0.0512 USD per Hour

On-Demand Windows base pricing: 0.0408 USD per Hour

Free tier eligible

All generations

Compare instance type

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login)Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

ansiblekey

Create new key pair

▼ Network settingsInfo

NetworkInfo

vpc-0538c231a226fe3c9

SubnetInfo

No preference (Default subnet in any availability zone)

Auto-assign public IPInfo

Enable

Firewall (security groups)Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Common security groupsInfo

Select security groups

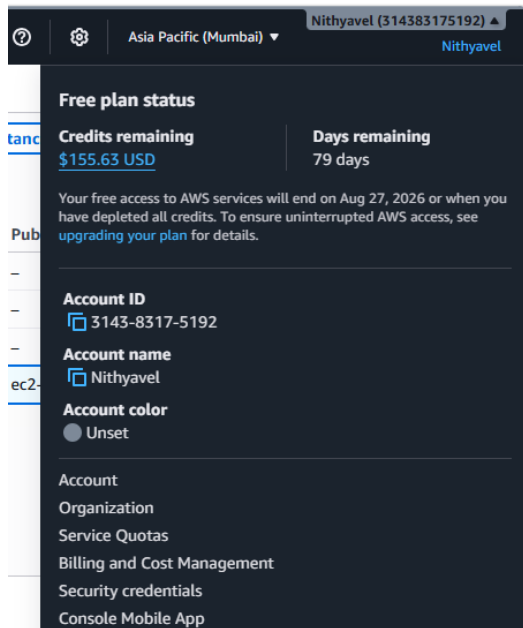
Compare security group rules

default sg-06d80e515c32b9515

VPC: vpc-0538c231a226fe3c9

Step no:3 In order to establish the communication between the EKS cluster and the EKS ec2 instance we need **access key and secrete access keys**.

Open security credentials



Create access keys

Access keys (1)						Actions	Create access key
Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. Learn more							
Access key ID	Created on	Access key last used	Region last used	Service last used	Status		
AKIAUSMVSRYMH74DOPV	22 days ago	8 days ago	us-east-1	s3	Active		

Alternatives to root user access keys [Info](#)

Root user access keys are not recommended
Root user access keys have long-term unlimited permissions that can't be restricted. Instead, when accessing AWS CLI, SDKs, or tools for local development with AWS, use the `aws login` command and your existing console credentials for access.
[Learn more about alternatives to root user access keys](#)

Continue to create access key?

☒ I understand creating a root access key is not a best practice, but I still want to create one.

[Cancel](#)

[Create access key](#)

Access key will be created

Retrieve access key [Info](#)

Access key

If you lose or forget your secret access key, you can create a new access key and make the old key inactive.

Access key

Secret access key Copied ***** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

Now connect our EKS ec2 instance and enter the command to configure the key: **aws configure**

Enter the access key and the secret access key.

Keep the regions default by just pressing enter.

```
[ec2-user@ip-172-31-8-71 ~]$ aws configure
AWS Access Key ID [None]: AKIAUSMVSRVYMLLTAOM7G
AWS Secret Access Key [None]: gZva9KGr2OVZtBZFOz+D1DrM8
Default region name [None]: Default output format [None]:
[ec2-user@ip-172-31-8-71 ~]$
[ec2-user@ip-172-31-8-71 ~]$
```

Step no:4 Now open our cluster we need to have worker nodes inside it in order to perform any tasks.

Our **cluster** is **active** now → click it → Go to **compute**

Clusters (1) Info				
Filter clusters				
	Cluster name	Status	Kubernetes version	
☐	mycluster	Active	1.35 Upgrade now	Info

Under **node groups**: click **Add**

Overview	Resources	Compute	Networking	Add-ons 1	Capabilities	Access	Observability	Update history & backups	Tags
Nodes (0) Info									
Filter Nodes by property or value									
Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage	
No Nodes This cluster does not have any Nodes, or you don't have permission to view them.									
Node groups (0) Info									
Node groups implement basic compute scaling through EC2 Auto Scaling groups.									
Filter node groups by property or value									
Group name	Desired size	AMI release version	Launch template	Status					

Configure the nodes here. Nodes are also ec2 instances / server.

So it will ask for the instance configurations for the worker nodes.

Name of the node group: **mynodegp**

Here also we need to create an **IAM role** → **create and select**.

Next

Configure node group [Info](#)

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add r

Node group configuration

These properties cannot be changed after the node group is created.

Name

Assign a unique name for this node group.

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters,

Node IAM role

To access other AWS services and resources your Nodes needs an IAM role. Choose a role with the right permissions for your No

✓ AmazonEKSAutoNodeRole successfully created.

AmazonEKSAutoNodeRole ▼



Create new role

[View role details in IAM](#) ↗

Launch template [Info](#)

These properties cannot be changed after the node group is created.

☐ Use launch template

Configure this node group using an EC2 launch template.

Type of AMI: **Amazon linux**

Capacity type: **On-demand**

Instance type: **Check the free tier available (c7i-flex.large)**

(If we provide other than free tier eligible our worker nodes will not be created)

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2023 (x86_64) Standard (AL2023_x86_64_STANDARD) ▼

Capacity type

Select the capacity purchase option for this node group.

On-Demand ▼

Instance types [Info](#)

Select instance types you prefer for this node group.

🔍 Enter an instance type

c7i-flex.large

vCPU: 2 vCPUs Memory: 4 GiB Network: Up to 12.5 Gigabit Max ENI: 3 Max IPs: 30



Disk size

Select the size of the attached EBS volume for each node.

20

GiB

For autoscaling → keep the **default** (maximum size can be increased, can not be decreased)

Node group scaling configuration

Desired size
Set the desired number of nodes that the group should launch with initially.

nodes

Desired node size must be greater than or equal to 0

Minimum size
Set the minimum number of nodes that the group can scale in to.

nodes

Minimum node size must be greater than or equal to 0

Maximum size
Set the maximum number of nodes that the group can scale out to.

nodes

Maximum node size must be greater than or equal to 1 and cannot be lower than the minimum size

AWS Auto Scaling Group (ASG) Warm Pool

☒ Use ASG warm pool
Reduce node provisioning time during scale-out events.

Networking also the default subnets.

Specify networking

Node group network configuration
These properties cannot be changed after the node group is created.

Subnets [Info](#)
Specify the subnets in your VPC where your nodes will run. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets

subnet-01bf4d44c0912db3
ap-south-1a 172.31.32.0/20

subnet-0c305b80f72541592
ap-south-1c 172.31.16.0/20

subnet-0b3c21b6de4314d95
ap-south-1b 172.31.0.0/20

Clear selected

☒ Configure remote access to nodes [Info](#)

Review and create.

mynodegrp

Node group configuration Info		
Kubernetes version 1.35	AMI type Info Amazon Linux 2023 (x86_64) Standard	Status Active
AMI release version Info 1.35.5-20260529	Instance types c7i-flex.large	Disk size 20 GiB

Once applied it will take few minutes to create the worker nodes. Once the creation is successful, we can see the two **worker nodes created on demand on the EC2 console page** with the configuration that we have provided. Wait until they are **ready**.

Instances (3) [Info](#)

Saved filter sets
Choose filter set

Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type
☐		i-00553ab4aab388e3f	Running	c7i-flex.large
☐	EKS	i-02a04addcdcae9efa3	Running	t3.small
☐		i-0d976c1e918c711ba	Running	c7i-flex.large

Step no:5 In our EKS instance (Communication instance) we need to make the connection with the two worker nodes we have created. The access keys and secret access keys are for authentication purpose for communication.

First **install the Kubernetes** in the communication node using the following command:

(git hub repo has this commands)

```
Last login: Wed Jun 10 13:32:27 2026 from 13.233.177.
[ec2-user@ip-172-31-8-71 ~]$ sudo su
[root@ip-172-31-8-71 ec2-user]# curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/linux/amd64/kubectl.tar.gz"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
kubectl version
```

Check the version: **kubectl version**

To establish the connection: **aws eks --region ap-south-1 update-kubeconfig --name mycluster**
(name and location of our cluster)

The nodes will be added

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl version
Client Version: v1.36.1
Kustomize Version: v5.8.1
The connection to the server localhost:8080 was refused - did you specify the right host or port?
[ec2-user@ip-172-31-8-71 ~]$ aws eks --region ap-south-1 update-kubeconfig --name mycluster
Added new context arn:aws:eks:ap-south-1:314383175192:cluster/mycluster to /home/ec2-user/.kube/config
[ec2-user@ip-172-31-8-71 ~]$
```

We can see the worker nodes from here: **kubectl get nodes**

```
added new context arn:aws:eks:ap-south-1:314383175192:cluster/mycluster
[ec2-user@ip-172-31-8-71 ~]$ kubectl get nodes
NAME                                                    STATUS    ROLES    AGE
ip-172-31-0-45.ap-south-1.compute.internal             Ready     <none>   82m
ip-172-31-42-214.ap-south-1.compute.internal           Ready     <none>   82m
[ec2-user@ip-172-31-8-71 ~]$
```

In worker nodes we must have pods to run any applications.

Create a node we must have a .yaml file for the node with its configurations and specifications.

Create a file: **vi pod.yaml**

Paste the pod contents. The pod has the containers to run the applications / servers.

```
ip-172-31-42-214.ap-south-1.compute.internal             Ready
[ec2-user@ip-172-31-8-71 ~]$ vi pod.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app-pod
  labels:
    app: my-app
spec:
  containers:
    # must be 'containers'
    - name: my-container
      image: httpd:latest
      ports:
        - containerPort: 80
```

Now create a pod from the pod.yaml file using: **kubectl apply -f pod.yaml**

We can see the pod using: **kubectl get pod**

```
CONTAINER ID   NAME          READY   STATUS    RESTARTS   AGE
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f pod.yaml
pod/my-app-pod created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod
NAME          READY   STATUS    RESTARTS   AGE
my-app-pod    1/1     Running   0           16s
[ec2-user@ip-172-31-8-71 ~]$
```

In order to expose the pod to outside world (to see the output) to run as an application we need a service.

Create a service we must have a .yaml file for the service with its configurations and specifications.

Create a file: **services.yaml**

We are using node port service. The node port range is 30000-32767 inside the TCP range.

```
[ec2-user@ip-172-31-8-71 ~]$ vi services.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat services.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  type: NodePort
  ports:
    - protocol: TCP
      port: 80      # Service port
      targetPort: 80 # Pod container port
      nodePort: 30007 # NodePort (must be between 30000-32767)
```

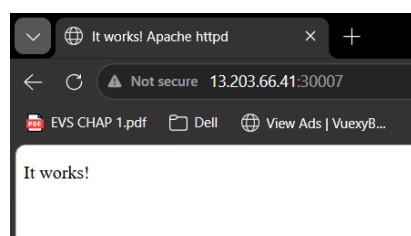
Now create a service from the services.yaml file using: **kubectl apply -f services.yaml**

We can see the pod using: **kubectl get service**

```
nodePort: 30007 # NodePort (must be between 30000-32767)
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f services.yaml
service/my-app-service created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get service
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes    ClusterIP   10.100.0.1    <none>        443/TCP          149m
my-app-service NodePort    10.100.235.66 <none>        80:30007/TCP     10s
[ec2-user@ip-172-31-8-71 ~]$
```

Now go to the worker nodes and check Default sg all TCP enabled. Http and httpd also because we are running this httpd inside the pod.

Copy the public Ip of any one of the worker nodes:30007 (node port provided in the services.yaml) and paste it in a private browser. We can see the httpd output page



We can see the respective instance in which the pod is running using: **kubectl delete pod my-app-pod**

```
my-app-service NodePort 10.100.235.66 <none> 80:30007/TCP 10s
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod -o wide
NAME READY STATUS RESTARTS AGE IP NODE
my-app-pod 1/1 Running 0 6m45s 172.31.39.112 ip-172-31-42-214.ap-south-1.comput
[ec2-user@ip-172-31-8-71 ~]$
```

The pods are ephemeral in nature. If we delete it, it will be dead. It won't create a new pod with this same configuration automatically to overcome the crash by self-healing.

Delete the pod: **kubectl delete pod pod-name**

Delete the service associated with the pod: **kubectl delete svc service-name**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl delete pod my-app-pod
pod "my-app-pod" deleted from default namespace
[ec2-user@ip-172-31-8-71 ~]$ kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
kubernetes ClusterIP 10.100.0.1 <none> 443/TCP
my-app-service NodePort 10.100.235.66 <none> 80:30007/TCP
[ec2-user@ip-172-31-8-71 ~]$ kubectl delete svc my-app-service
service "my-app-service" deleted from default namespace
```

Step no:6 In order to overcome the ephemeral nature of pod, we have deployments which has replica set inside it. If the deployment crashes, it will auto heal and try to create a new one based on the number of replica sets in the .yaml File

Create a deployment file: **vi deployment.yaml** (it has labels: object and selectors (filter to match the labels))

Similarly, the deployment runs the containers.

```
[ec2-user@ip-172-31-8-71 ~]$ vi deployment.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

Apply the deployment.yaml file to create a deployment: **kubectl apply -f deployment.yaml**

Once the deployment has been created. Check the pod info → we can see 5 pods are running (replicas:5 in the deployment.yaml)

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f deployment.yaml
deployment.apps/my-app-deployment created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP
my-app-deployment-58f59c99c-8w5jb  1/1     Running   0           21s   172.31.9.209
my-app-deployment-58f59c99c-dh4rh  1/1     Running   0           21s   172.31.42.95
my-app-deployment-58f59c99c-nvxmj  1/1     Running   0           21s   172.31.46.25
my-app-deployment-58f59c99c-nzs85  1/1     Running   0           21s   172.31.13.110
my-app-deployment-58f59c99c-tp6lb  1/1     Running   0           21s   172.31.32.79
[ec2-user@ip-172-31-8-71 ~]$
```

Delete one of the pods in deployment manually and check the pods. We can see a new one has been autoscaled immediately to match the replica numbers. (within 6s)

```
my-app-deployment-58f59c99c-tp6lb  1/1     Running   0           21s   172.31.32.79
[ec2-user@ip-172-31-8-71 ~]$ kubectl delete pod my-app-deployment-58f59c99c-8w5jb
pod "my-app-deployment-58f59c99c-8w5jb" deleted from default namespace
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP
my-app-deployment-58f59c99c-dh4rh  1/1     Running   0           2m49s  172.31.42.95
my-app-deployment-58f59c99c-nvxmj  1/1     Running   0           2m49s  172.31.46.25
my-app-deployment-58f59c99c-nzs85  1/1     Running   0           2m49s  172.31.13.110
my-app-deployment-58f59c99c-tp6lb  1/1     Running   0           2m49s  172.31.32.79
my-app-deployment-58f59c99c-tz5f5  1/1     Running   0           6s     172.31.8.128
[ec2-user@ip-172-31-8-71 ~]$
```

In order to expose the pods inside the deployment to outside world (to see the output) to run as an application we need a service. Create a service we must have a .yaml file for the service with its configurations and specifications.

Create a file: **vi service.yaml**

We are using **Load balancer service**. Make sure that we have deleted the existing service running.

```
[ec2-user@ip-172-31-8-71 ~]$ vi service.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80          # Service port
      targetPort: 80    # Pod container port
```

Apply the service: **kubectl apply -f service.yaml**

Check the list of service available. We can see our LoadBalancer service.

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f service.yaml
service/my-app-service created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get service
NAME                TYPE           CLUSTER-IP      EXTERNAL-IP
kubernetes          ClusterIP      10.100.0.1       <none>
my-app-service      LoadBalancer   10.100.5.218     aabc7e0e6a49b4
[ec2-user@ip-172-31-8-71 ~]$
```

Step no:7 A load balancer will be created for managing these pods in the deployment.

Go to **Ec2 console** → **load balancers** → click on **the LB automatically created**.

Load balancers (1/1) [What's new?](#)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	State	Type	Scheme
-------------------------------------	------	-------	------	--------

☒	aabc7e0e6a49b490fb32f8dd0c5f1180	-	classic	-
-------------------------------------	----------------------------------	---	---------	---

Security groups → **Edit rule** → **Inbound rules** → **Add all TCP** (check HTTP, ICMP4 present, if not add them also)→save.

[Security Groups](#) > [sg-0d287d41471b7f7fa - k8s-elb-aabc7e0e6a49b490fb32f8dd0c5f1180](#) > [Edit inbound rules](#)

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules [Info](#)

Security group rule ID

sg-0936376367694ce81

Type [Info](#)

Custom ICMP - IPv4

Protocol [Info](#)

Destina...

Port range [Info](#)

fragme...

Source [Info](#)

Custom

Q

0.0.0.0/0 X

sg-0552e6952629dba63

HTTP

TCP

80

Custom

Q

0.0.0.0/0 X

-

All TCP

TCP

0 - 65535

Anywhe...

Q 0.0.0.0/0

0.0.0.0/0 X

Add rule

Copy the DNS (endpoint url) of the load balancer and paste in a private browser. We can see the nginx page as we are running nginx inside the pods inside the deployment.

aabc7e0e6a49b490fb32f8dd0c5f1180

▼ Details

Load balancer type

Classic

Status

2 of 2 instances in service

Scheme

Internet-facing

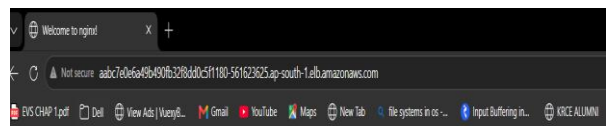
Hosted zone

ZP97RAFLXTNZK

✔ DNS name copied

aabc7e0e6a49b490fb32f8dd0c5f1180-561623625.ap-south-1.elb.amazonaws.com (A Record)

ⓘ This Classic Load Balancer can be migrated to a next generation load balancer. Migration wizard uses your I



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to [nginx.org](#).
To engage with the community please visit [community.nginx.org](#).
For enterprise grade support, professional services, additional security features and capabilities please refer to [fs.com/nginx](#).

Thank you for using nginx.